UCS503 – Software Engineering

Thapar Exchange

Campus Marketplace and Resource Sharing Platform

Submitted by:

1024031005 Kanishk Khinchi
1024030290 Mahesh
1024030729 Amrita

Group: 3C53, BE Third Year, C0E

Submitted to:

Dr. Paramveer Sidhu

Project Evaluation Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 4
- 1.3 Scope and Limitations . . . 4

2 Methodology and Design **5**
- 2.1 System Architecture . . . 5
  - 2.1.1 High-Level Workflow . . . 6
  - 2.1.2 Technical Stack . . . 6
- 2.2 Implementation Details . . . 7
  - 2.2.1 Authentication and Listing Management . . . 7
  - 2.2.2 Rental Booking Workflow . . . 7
  - 2.2.3 Database Design . . . 7

3 Results and Evaluation **9**
- 3.1 Environment and Database Setup . . . 9
- 3.2 Functional Test Plan . . . 9
- 3.3 Performance Evaluation . . . 10
- 3.4 Analysis . . . 11
- 3.5 Limitations . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations . . . 12
- 4.4 Final Remarks . . . 13

Chapter 1

Introduction

1.1 Background and Context

Students often need resources for a limited period, including textbooks, calculators, cycles, sports equipment, electronics, hostel essentials, and other useful items. Buying every item new can be expensive and unnecessary when a resource is needed temporarily. At the same time, students may own items that are no longer in regular use but could be useful to others.

Such exchanges are often arranged through messaging groups, personal contacts, or informal social media posts. These approaches can make it difficult to discover relevant items, compare details, confirm availability, and maintain a structured record of rental requests. Thapar Exchange, also referred to as ThaparRent in the repository README, is a campus-focused web application intended to make student-to-student resource sharing more organized.

The repository contains a React-based frontend and an Express backend connected to a PostgreSQL database through Prisma ORM. The current backend includes account registration and login, listing endpoints, and rental-booking workflows. The project is under active development; therefore, this report distinguishes implemented functionality from broader features proposed for future versions.

1.1.1 Problem Statement

Students at Thapar Institute of Engineering and Technology may need to rent or share items, but informal channels do not provide a single structured place to publish listings, browse resources, authenticate users, and manage rental requests. Information can be scattered across different groups, while requests and their statuses may be difficult to track consistently.

The project addresses this problem through a web-based campus marketplace with user accounts, item listings, and a database-backed booking process. The current backend supports registration and login, public listing retrieval, authenticated listing creation, and booking requests with owner-controlled acceptance or rejection. Institutional email verification, advanced search filters, condition documentation, reporting, and administrative moderation should not be considered complete unless verified in the current code.

1.2 Project Objectives

1. Develop a centralized web application for students to discover and offer campus resources.
2. Implement registration and login, storing password hashes rather than plaintext passwords.
3. Provide listing endpoints for item details, category, daily rental price, optional image URL, availability, and owner.
4. Implement rental requests and owner-controlled acceptance or rejection.
5. Store users, listings, and bookings in PostgreSQL using Prisma ORM.
6. Validate core workflows and document limitations before making performance, usability, or security claims.

1.3 Scope and Limitations

This report focuses on the repository's architecture, backend modules, data model, and local database setup. The observed setup results are that Prisma schema validation succeeded, Prisma Client was generated, and the existing initialization and booking migrations were applied successfully to the local PostgreSQL database. This does not independently establish that every end-to-end user workflow works.

No verified performance benchmarks, formal usability-study results, or evidence that all planned trust and moderation features are complete were supplied. These are treated as evaluation tasks or future work rather than implemented outcomes.

Chapter 2

Methodology and Design

2.1 System Architecture

The application follows a client–server architecture. The frontend uses React and Vite [**reactdocs**; **vitedocs**]. It communicates with a backend API implemented using Node.js and Express [**expressdocs**]. The backend uses Prisma ORM to access a PostgreSQL database [**prismadocs**; **postgresdocs**]. During local development, the Vite configuration proxies API requests to the backend.

Figure 2.1 presents the high-level component arrangement. Requests originate in the browser-based frontend, are handled by Express routes and middleware, and access persistent data through Prisma.

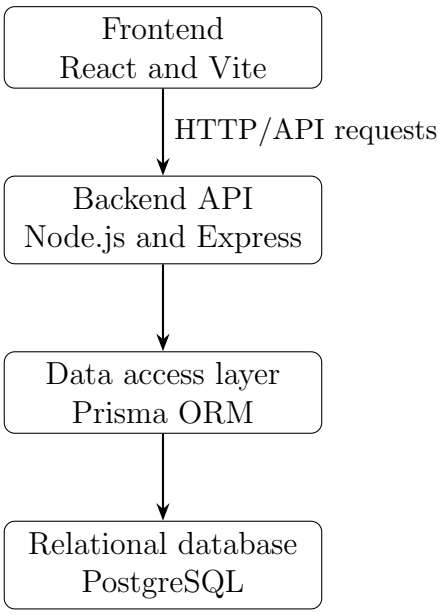


Figure 2.1: High-level architecture of Thapar Exchange

The main entities are users, listings, and bookings. A user can own multiple listings, and a listing can have multiple booking records. Each booking identifies a listing and a renter, and stores a status and timestamps. Listing states are `AVAILABLE` and `UNAVAILABLE`; booking states are `PENDING`, `ACCEPTED`, `REJECTED`, and `CANCELLED`.

2.1.1 High-Level Workflow

1. A student registers using a name, email address, and password, or logs in with existing credentials.

2. The backend validates the request, hashes new passwords with bcrypt [**bcrypt**], and issues a JSON Web Token (JWT) after successful authentication [**jwt**].

3. Public listing endpoints allow clients to retrieve listings and listing details.

4. An authenticated student creates a listing. The backend derives the owner identity from the authentication token rather than trusting an owner ID supplied in the request body.

5. A renter submits a booking request for an available listing. The backend prevents a user from requesting their own listing and rejects duplicate pending requests for the same listing by the same renter.

6. The listing owner can accept or reject a pending booking. The booking status is persisted in the database.

2.1.2 Technical Stack

Table 2.1: Technologies used in the current repository

Technology	**Role in the project**
JavaScript	Main language used by the frontend and backend.
React	Browser-based user interface.
Vite	Development server and frontend build tooling.
Node.js	Runtime for the backend API.
Express 5	HTTP routes and middleware.
PostgreSQL	Relational database for users, listings, and bookings.
Prisma ORM	Database schema, client access, and migrations.
bcrypt	Password hashing.
JSON Web Token (JWT)	Authentication tokens for protected API requests.
Git	Version control for source files and project history.

2.2 Implementation Details

2.2.1 Authentication and Listing Management

The authentication routes are implemented in `backend/src/auth.routes.js`. Registration accepts a name, email, and password. The backend hashes the password using bcrypt and stores the hash in the user record. Login checks submitted credentials and returns a JWT and public user information. The middleware in `backend/src/auth.middleware.js` protects routes that require authentication.

Listing functionality is implemented in `backend/src/listings.routes.js`. The database schema defines title, description, category, daily price, optional image URL, status, owner ID, and timestamps. Listing retrieval endpoints are public, while listing creation requires authentication. The owner ID is derived from the authenticated user, helping prevent clients from creating listings on behalf of another account.

2.2.2 Rental Booking Workflow

Booking functionality is implemented in `backend/src/bookings.routes.js`. An authenticated renter can request an available listing. The renter identity is derived from the token, and a user cannot request their own listing. The backend also rejects a duplicate pending request by the same renter for the same listing.

A listing owner can retrieve booking requests associated with their listings and change a pending booking to `ACCEPTED` or `REJECTED`. These controls provide a basic structured rental-request workflow; they do not establish that physical handover, item-condition verification, payment, dispute resolution, or return confirmation has been implemented.

2.2.3 Database Design

The Prisma schema defines three main models:

- **User:** Unique email address, name, password hash, related listings, related bookings, and timestamps.
- **Listing:** Item details, daily price, optional image URL, availability status, owner relationship, and timestamps.
- **Booking:** Listing and renter relationships, request status, and timestamps. Indexes support queries by listing/status and renter/creation time.

Figure 2.2 presents a conceptual view of the entities and relationships. Exact fields and constraints are defined by the repository's Prisma schema.

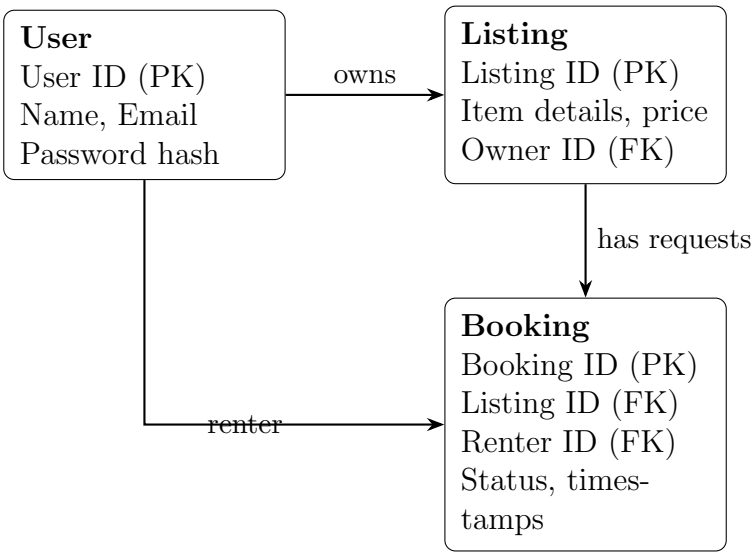


Figure 2.2: Conceptual entity–relationship model

The backend README documents configuration through a `DATABASE_URL` connection string and a private `JWT_SECRET`. Credentials should be stored in a local environment file and must not be committed to the repository. Database migrations and Prisma Client generation must run against a correctly configured PostgreSQL instance.

Chapter 3

Results and Evaluation

3.1 Environment and Database Setup

The local database setup was performed using PostgreSQL and Prisma. The following outcomes were observed in the development terminal:

- Prisma schema validation completed successfully.
- Prisma Client generation completed successfully.
- The existing initialization migration and booking-related migration were applied.
- Prisma reported that the database was in sync with the schema.

These outcomes verify database configuration and migration execution in the local development environment. They do not prove that all API endpoints or frontend workflows have passed functional testing.

3.2 Functional Test Plan

The following test cases should be executed and their observed results recorded before they can be marked as passed.

Table 3.1: Functional test cases and expected outcomes

Test area	**Test case**	**Expected outcome**
Database	Validate schema and apply migrations	Schema valid; migrations applied
Authentication	Register with valid details	Account created and password hashed
Authentication	Login with incorrect credentials	Request rejected
Authentication	Access protected route without token	Request rejected
Listings	Retrieve public listings	Listing data returned
Listings	Create listing without authentication	Request rejected
Bookings	Request own listing	Request rejected
Bookings	Submit duplicate pending request	Duplicate request rejected
Bookings	Owner updates pending request	Status changes to accepted or rejected
Integration	Retrieve records after create/update	Persisted state is returned
Frontend	Use UI while backend is running	UI handles success/error states

The database setup cases above were observed as successful. The remaining functional cases are test recommendations until their execution and results are recorded. No unsupported pass rate is reported.

3.3 Performance Evaluation

The available repository and terminal output do not provide verified measurements of API response time, throughput, concurrent-user capacity, or user-study outcomes. Numerical performance claims are therefore not made. Future evaluation should record:

- **API response time:** Median and 95th-percentile response times under a documented workload.
- **Authentication correctness:** Successful and rejected registration/login cases.
- **Listing workflow correctness:** Whether valid operations succeed and invalid requests are rejected.
- **Booking workflow correctness:** Ownership, availability, duplicate pending requests, and status transitions.
- **Database consistency:** Related records after create and update operations.

- **Usability feedback:** Student completion of listing discovery and booking tasks, with feedback recorded.

3.4 Analysis

The repository establishes a backend foundation for a campus rental platform. It includes authentication, protected listing creation, public listing retrieval, booking requests, ownership checks, status transitions, and a relational data model. The separate frontend, API server, and database layer provide a modular structure for further development.

Features such as richer search and filtering, institutional student verification, condition documentation, reporting, and administrative moderation should not be treated as complete unless confirmed in the current code and tested in the running application. The available source and migration output do not provide measured evidence of system performance or student acceptance. A complete evaluation requires repeatable API tests, frontend integration checks, and feedback from target users.

3.5 Limitations

- Only schema validation, client generation, and migration execution are confirmed by the available terminal output.
- Functional API and end-to-end frontend tests have not been documented here as completed.
- No performance benchmark, load test, or usability study has been supplied.
- Institutional verification, moderation, condition records, payments, and dispute management require separate implementation and verification where applicable.
- Successful local setup does not establish production readiness; deployment, secret management, logging, validation, authorization, and security controls need further review.

Chapter 4

Conclusion

4.1 Summary of Work

Thapar Exchange is a campus marketplace and resource-sharing project intended to help students discover and offer useful items through a structured web platform. The repository contains a React frontend, a Node.js and Express backend, and a PostgreSQL database accessed through Prisma ORM. The backend supports account registration and login, listing retrieval and creation, and a rental booking workflow with owner-controlled acceptance or rejection.

The local database setup was completed successfully: the schema validated, the Prisma Client was generated, and the existing migrations were applied. The next step is to test the application end to end so that the working status of each user-facing workflow can be documented accurately.

4.2 Key Findings

- The repository uses a modular client–server architecture with a relational database layer.
- Password hashing and JWT-based authentication are used in the backend authentication flow.
- Listing and booking routes include ownership and request-state checks, including prevention of duplicate pending requests.
- Database schema validation, client generation, and migration execution succeeded in the local environment.
- Broader trust, moderation, and condition-documentation features require verification before being described as complete.
- Numerical performance claims should be added only after documented measurements have been collected.

4.3 Future Work and Recommendations

1. Complete and validate frontend workflows for registration, login, listing creation, listing discovery, and booking management.

2. Implement and verify institutional email validation if access should be restricted to eligible TIET students.
3. Add search and filtering by category, price, availability, and pickup location where not already implemented.
4. Introduce item-condition documentation and reporting mechanisms to improve transparency and trust.
5. Develop moderation and administration tools for reported listings and inappropriate content.
6. Add automated tests for authentication, listing permissions, booking status transitions, and database consistency.
7. Conduct usability testing with students and benchmark API response times under a documented workload.
8. Review deployment configuration, secret management, input validation, logging, and access control before real-world launch.

4.4 Final Remarks

Thapar Exchange addresses a practical campus need: making it easier for students to find and share resources. The current repository provides core backend building blocks for users, listings, and rental requests, together with a modern web development stack. The next stage should focus on verifying end-to-end functionality, completing important trust and usability features, and collecting documented test evidence. The project should be presented as an evolving academic software system, with implemented capabilities and future plans clearly distinguished.